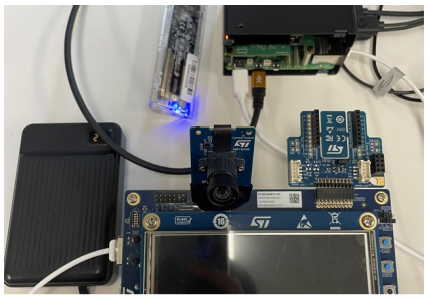
제 24 회 임베디드 SW 경진대회 · 모빌리티 부문

개발완료보고서

IcePredict

AI 예측과 RTOS 반응의 이중 안전망 블랙아이스 대응 시스템

카메라가 블랙아이스를 미리 보고, 놓치면 노면 마찰을 직접 재서 제때 세운다.



STM32N6570-DK + 라즈베리파이 5 + AI 카메라



Moza 휠·페달로 직접 운전하는 HIL



2차 방어: 빙판 마찰을 재서 미리 제동

표 1. 팀 정보

항목	내용
팀명	ARTS
팀원	남윤상(팀장) · 김진찬 · 이지성
작품명	IcePredict
주요 구성	STM32N6570-DK(Neural-ART NPU · ThreadX) · Raspberry Pi 5 + AI HAT+ 2 · CARLA HIL · Moza 휠·페달
소스 코드·근거	https://github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS

목차

1. 개요
2. 문제 정의 — 왜 블랙아이스는 카메라만으로 풀리지 않는가
3. 시스템 구성
4. 개발계획서 대비 변경점
5. 1 차 방어 — 카메라가 미리 본다
6. 2 차 방어 — 카메라가 놓치면 차량 거동으로 받는다
7. 2 차 방어 확장 — 노면 마찰을 재서 제동 시점을 고친다
8. 안전 전용 RTOS — 메인 컴퓨터가 멈춰도 2 차 방어는 산다
9. 시연 구성
10. 검증 방법과 재현성
11. 한계와 향후 계획
부록. 근거 문서 목록

1. 개요

IcePredict 는 블랙아이스 사고를 줄이는 **운전자 보조 장치**다. 조향은 운전자에게 두고 경고·가속 제한·제동만 돕는다. 두 겹의 방어로 구성된다. **1 차 방어**는 카메라 AI 가 노면을 보고 빙판에 닿기 전에 경고하고 감속한다. **2 차 방어**는 카메라가 놓쳤을 때 IMU 로 차량 거동을 보고 미끄러짐을 확정하며, 확장된 2 차 방어는 노면 마찰을 직접 재서 '언제부터 서야 하는지'를 노면에 맞게 다시 계산한다. 2 차 방어 판단은 메인 컴퓨터(리눅스)와 분리된 **안전 전용 RTOS** 에서 돌아 메인 컴퓨터가 멈춰도 계속된다.

표 2. 핵심 결과

구분	결과	조건
1 차 방어	블랙아이스 경보율 **96.6 %**, 마른 노면 오경보율 **0.3 %**	실제 도로 사진 69,358 장을 STM32N6 보드 NPU 에 직접 입력
2 차 방어 응답	최악 **22.6 µs** (리눅스 최악 5,790 µs)	STM32N6 + ThreadX, 49,405 표본
2 차 방어 효과	방어 없음 9/9 제어 상실 → 2 차 방어 차선 안 정지	CARLA HIL, 같은 빙판·날씨·주행선
마찰 인식 제동	정지 **2/9 → 8/9** (7 회는 목표대로 앞차 4 m 앞)	CARLA, 빙판 위 정지 차량, 마찰 3 종 × 속도 3 종
고장 격리	메인 컴퓨터 정지 시 충돌 속도 **53 → 39 km/h**	곡선 빙판 µ 0.12 · 60 km/h
실시간성	전 코어 부하에서 20 ms 주기 6,000 회 마감 놓침 **0** (최악 0.06 ms)	파이 격리 코어(안전 전용 RTOS 대체)

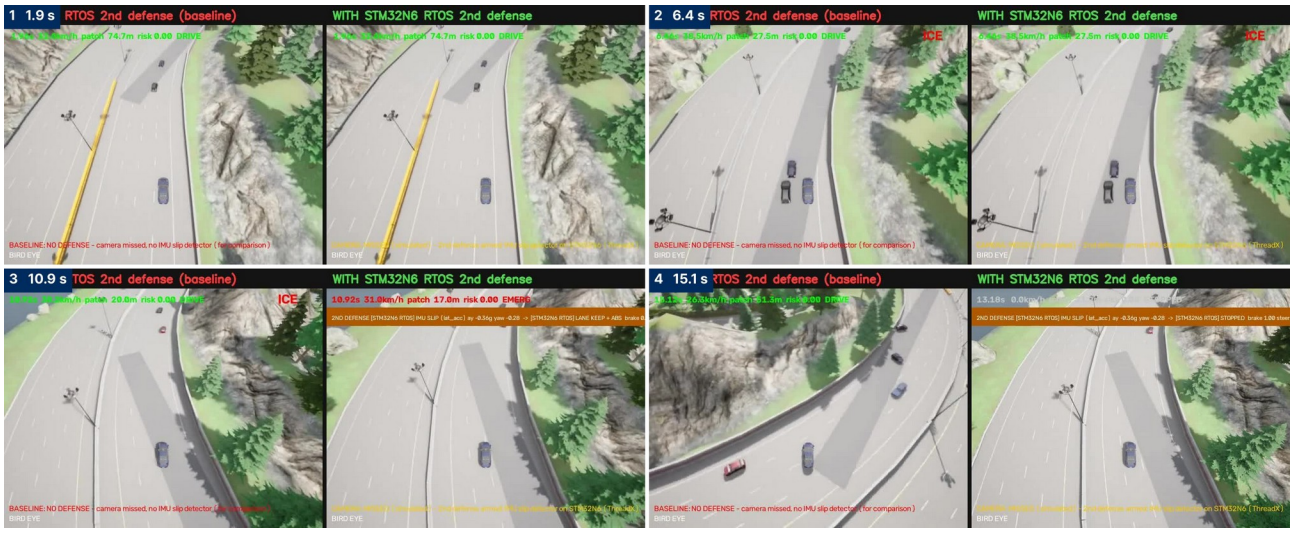


그림 1. 같은 조건 비교(조감) — 각 칸의 왼쪽은 방어 없음(제어를 잃고 미끄러짐), 오른쪽은 STM32N6 RTOS 2 차 방어(차선 안 정지). 시간 순서 4 장면

2. 문제 정의 — 왜 블랙아이스는 카메라만으로 풀리지 않는가

- **라벨이 없다.** 공개 데이터셋에 '블랙아이스' 라벨이 없다. RSCD 의 ice 57,262 장도 다져진 눈·서리에 가까운 '얼음 노면'일 뿐, 투명한 블랙아이스를 따로 구분하지 않는다.
- **원래 어려운 문제다.** 투명·검은 얼음은 아스팔트와 시각적으로 구분되지 않는다. 우리 모델도 실사진 얼음 6,340 장 중 215 장(3.4 %)을 어떤 문턱으로도 잡지 못했다.
- **지금 차의 빈틈.** ABS·ESC 는 미끄러지는 바퀴를 잡지만 노면 마찰을 다른 장치에 알리지 않는다. 긴급제동·간격 유지는 앞차 거리로 제동 시점을 정하는데, 마찰을 모르면 마른 노면 기준이라 빙판에서 늦다. 60 km/h 에서 마른 노면은 38 m 앞에서 밟으면 서지만 블랙아이스(µ 0.16)는 105 m 가 필요하다.

그래서 1 차는 블랙아이스를 직접 맞히려 하지 않고 '결빙 위험 노면 확률 + 반사도 이상 + 기상 맥락'으로 위험도를 올린다. 그 전략이 실패할 때를 위해 2 차가 있다.

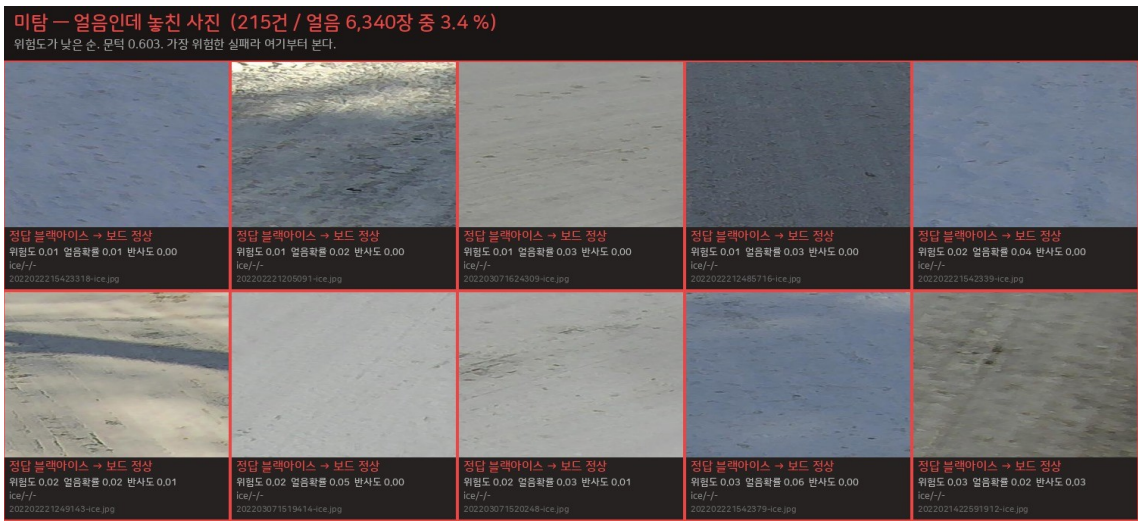


그림 2. 보드가 실제로 놓친 실사진 — 정답은 모두 블랙아이스인데 '정상'으로 판정했다(위험도 0.01~0.04)

3. 시스템 구성

3.1 이중 안전망 구조

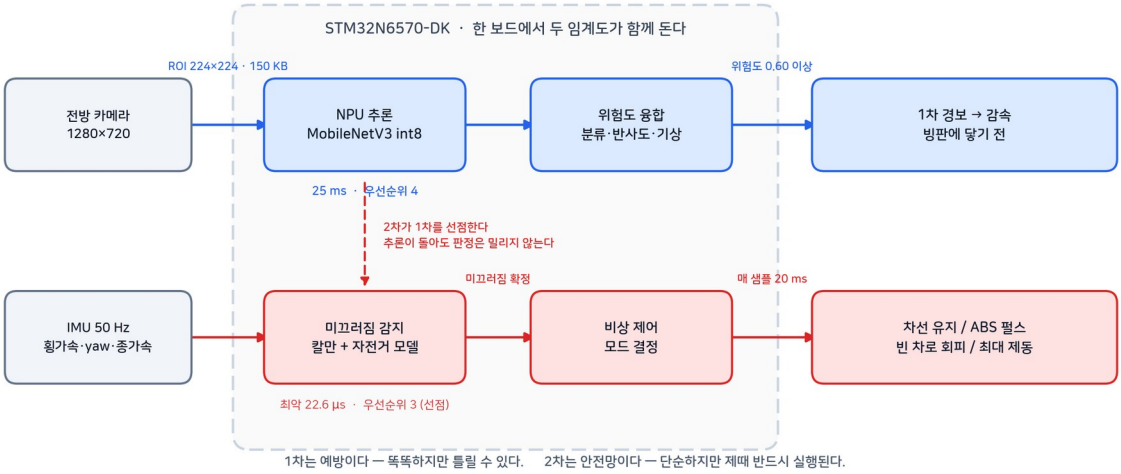


그림 3. 시스템 구조 — STM32N6 한 보드에 1 차(NPU)와 2 차(ThreadX)가 공존하고, 라즈베리파이가 기상 맥락·브리지를 맡는다

표 3. 두 방어의 역할

	1 차 방어 · 예측	2 차 방어 · 반응
센서	전방 카메라	IMU (횡가속 · yaw rate · 종가속)
판단	노면 4 분류 + 반사도 + 기상 맥락 → 위험도	칼만 필터 + 자전거 모델 잔차 → 미끄러짐 확정, 노면 확인 탭 → 마찰 → 제동 시점
시점	빙판에 닿기 전	카메라가 애매할 때·미끄러지기 시작한 뒤
실행	STM32N6 NPU · 프레임당 25 ms	STM32N6 + ThreadX · 최악 22.6 µs

ThreadX 우선순위는 IMU 융합 스레드가 3, NPU 프레임 스레드가 4 다. 25 ms 짜리 추론이 돌고 있어도 IMU 판정이 선점한다(혼합 임계도 구조).

3.2 하드웨어

표 4. 하드웨어 구성

구분	정식 명칭	역할
메인 보드	STM32N6570-DK (MB1939-N6570-	1 차 NPU 추론(Neural-ART @ 1 GHz) + 2 차 ThreadX 실시

	C02 + MB1860B)	간 판정·제어
카메라	ST AI Camera module MB1854B	1 차 방어 입력. 보드에 직결되어 NPU 로 바로 들어간다
호스트	Raspberry Pi 5 16 GB + AI HAT+ 2 (Hailo-10H)	기상 맥락 생성, ZMQ 브리지 (대회 필수 보드)
센서	Intel RealSense D435i	RGB·뎁스·IMU — 1 차 방어 카메라 시연
운전 장치	Moza R3 휠 + SRP 페달	사람이 직접 CARLA 를 운전하는 HIL
렌더	RTX 5090 · RTX 3090 워크스테이션	CARLA 0.9.16 물리·센서
저장·전원	NVMe 1 TB SSD · 보조배터리	데이터·로그 / 콘센트 없이 같은 구성

4. 개발계획서 대비 변경점

표 5. 변경점과 이유

항목	계획서(6 월)	실제 구현	바꾼 이유
2 차 실행 환경	Pi 5 + 리눅스	**STM32N6 + ThreadX**	리눅스 최악 깨어남 지연 5,790 µs — 20 ms 제어 주기의 29 % 를 지터 한 번이 먹는다
1 차 NPU	Hailo-10H 단독	**Neural-ART(N6) 주**	Hailo Dataflow Compiler 미확보. 1·2 차를 한 보드에 올려 선점 구조를 증명
검증	1/5 차량 저마찰 실측	**HIL + 실사진 69,358 장**	실물 빙판 재현이 어렵다. 펌웨어는 실물 보드, 비전은 실제 도로 사진으로 측정
2 차 출력	미끄러진 뒤 제동·조향	**+ 마찰 인식 제동 시점**	실험 결과 결과를 정한 것은 제동 시작 시점이었다 (7 장)

5. 1 차 방어 — 카메라가 미리 본다

5.1 데이터 전략

데이터셋을 먼저 고르지 않았다. 모델의 각 헤드가 무엇을 정답으로 배워야 하는지를 정의한 뒤 공개 데이터셋을 매핑했다. 주 학습은 RSCD(27 클래스 → 정상·젖음·결빙 위험·포트홀 4 클래스)이고, 반사도 헤드는 RoadSaW 의 실측 수막 두께, 국내 도로 검증은 AI Hub, 2 차 칼만 공분산은 PVS 실측 IMU 를 근거로 했다. 보조로 ROAD·CADC·WADS 를 썼다.

표 6. 모델 헤드별 정답과 데이터셋

방어	필요 역할	필요한 정답	데이터셋	활용
1 차	노면 상태 분류	건조/젖음/눈/결빙 클래스	**RSCD**	주 학습
1 차	반사도·젖음 정도	실측 수막 두께	**RoadSaW**	반사도 헤드 근거
1 차	국내 도로·악천후	눈·비·안개 + 조도	**AI Hub**	도메인 검증
2 차	IMU 미끄러짐 감지	실측 6 축 IMU + 노면 라벨	**PVS**	칼만 필터 검증
2 차	카메라-IMU 융합	RGB-IMU 동기, 악조건	**ROAD**	비교 기준선
보조	폭설 실주행	눈 장면·포인트 라벨	**CADC · WADS**	악천후 점검
보조	차선 가시성	자동 라벨	**CARLA**	합성 데이터 생성

RSCD 는 약 100 만 장(공개 서브셋 37 만 장)의 실도로 노면 패치로, 27 클래스(마찰 6 × 재질 4 × 요철 3)를 우리 4 클래스로 묶었다 — 정상 ← 건조, 젖음 ← 젖음·물 고임, 결빙 위험 ← 결빙·녹은 눈, 포트홀 ← 요철 '심함'. RoadSaW 에는 눈·얼음이 없고 ROAD 의 클래스는 노면 종류라 결빙이 없다 — 이 한계를 알고 역할을 나눠 썼다.



그림 4. 주요 데이터셋 — RSCD 클래스별 샘플, RoadSaW 젖음 단계, PVS IMU 장착 (각 데이터셋 논문·공식 자료에서 인용)

5.2 모델과 보드 배포

표 7. RoadNet 과 int8 배포

항목	값
백본	MobileNetV3-Small (ImageNet 사전학습)
입력 / 출력	1×3×224×224 int8 / 4 클래스 로짓 + 반사도 헤드
양자화	int8 PTQ(QDQ), stem 등화 + 클리핑으로 정확도 회복, u8→i8 변환
NPU	Neural-ART @ 1 GHz, 추론 25 ms
카메라 ROI	전방 7.9 ~ 42.2 m (FOV 60°, 피치 −12°)

5.3 위험도 융합

risk = α · p_ice + β · 반사도 + ɣ · (1 − 차선 가시성). 기상·위치 맥락(교량·새벽 등)이 가중치를 바꾼다. 쓸 수 없는 신호는 0 을 넣지 않고 분모에서 빼서 재정규화한다(0 은 중립이 아니라 최솟값이라 상한이 잘린다). 문턱은 초기 0.441 에서 실사진 전량 스윕으로 **0.603** 으로 재교정해 젖은 노면 오경보를 40 % 에서 0.8 % 로 내렸다.

표 8. 기상·위치 맥락에 따른 융합 가중치

맥락	α (분류)	β (반사도)	ɣ (차선)
교량 · 새벽 등 결빙 위험 높음	0.35	**0.45**	0.20
일반	**0.50**	0.30	0.20
저위험	**0.55**	0.15	0.30

폭우에서는 빙판이 없는 대조군에서도 위험도가 0.96 까지 올라 어떤 문턱으로도 막을 수 없었다. 그래서 강수 게이트로 1 차 방어를 스스로 끄고 2 차에 맡긴다 — 못 막을 경보를 내는 것보다 '지금은 못 본다'고 말하는 편이 낫다.



그림 5. 맥락 계층이 보는 입력 — 같은 빙판·같은 주행선에서 날씨만 바꿨다

5.4 실사진 대규모 평가

표 9. 실제 도로 사진 69,358 장 — STM32N6 보드 판정

실제 노면	장수	정답률	경보율(문턱 0.603)
블랙아이스	6,340	96.1 %	**96.6 %**
마른 노면	19,018	80.7 %	**0.3 %**
젖은 노면	34,440	84.6 %	**0.8 %**
포트홀	9,560	90.2 %	0.3 %

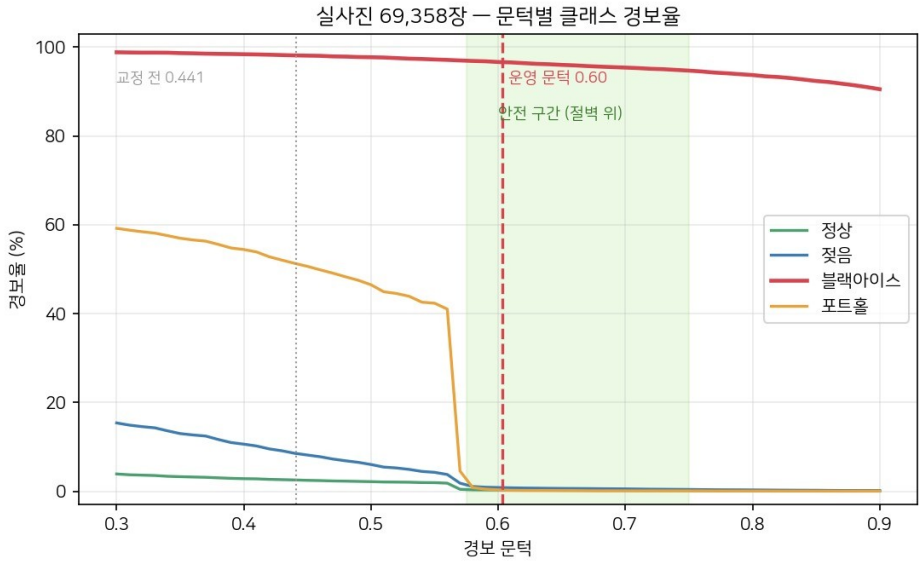


그림 6. 운영 문턱 결정 — 69,358 장 전량 스윕에서 절벽 구간을 찾아 0.603 으로 정했다

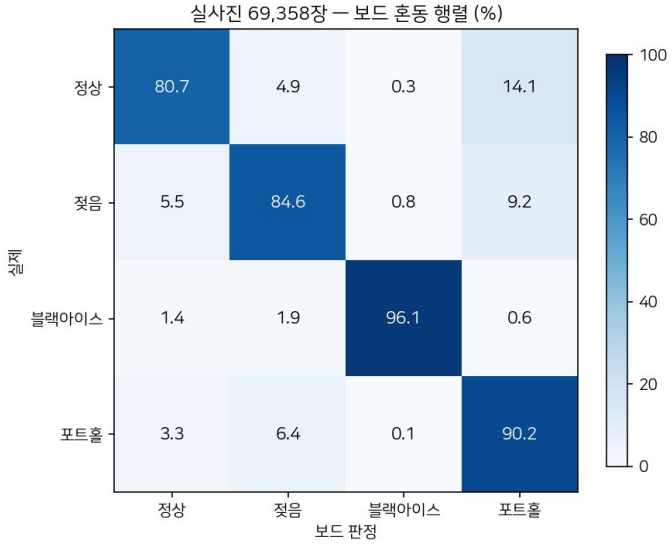


그림 7. 혼동 행렬 (행 = 실제 노면, 열 = 보드 판정)

5.5 CARLA 동작

CARLA Town04, 맑은 낮, 40 km/h, 마찰 0.02. 6.10 s 에 위험도 0.443 으로 1 차 경보, 7.14 s 에 빙판 23.6 m 앞에서 정지했다. 같은 주행선에서 날씨 11 종을 모두 돌려 인식·정지 14 회, 경고 늦음 7 회, 미인식 10 회, 오경보 1 회였다.



그림 8. 1 차 방어 동작 — 각 칸 왼쪽 전방 카메라(NPU 입력), 오른쪽 조감. 접근 → 경보 → 빙판 앞 정지



그림 9. 날씨 11 종에서 1 차 경보가 난 같은 순간 — 같은 주행선·같은 빙판에서 날씨만 바꿨다

6. 2 차 방어 — 카메라가 놓치면 차량 거동으로 받는다

6.1 미끄러짐 판정

· IMU 50 Hz 에서 횡가속 a_y · yaw rate · 종가속을 받는다.

· 2 상태 칼만 필터(값 · 변화율)로 잡음을 거른다.

· 자전거 모델로 '이 속도·이 조향이면 나와야 할 yaw rate'를 계산한다(고속 언더스티어 보정, 조향 지연 τ 0.06 s).

· 실측과 모델의 잔차가 **타원 밖**이면 미끄러짐 — 3 샘플(60 ms) 연속이면 확정한다. 직사각형 규칙은 모서리에서 저속 지연이 있어 타원으로 바꿨다.

표 10. 미끄러짐 판정 파라미터

항목	값	이유
축거 L	2.7 m	자전거 모델
조향 지연 τ	**0.06 s**	급조향 때 '모델이 즉답한다'는 가정이 오탐을 만들었다
횡가속 · yaw 오차 임계	0.30 g · 0.35 rad/s	타원의 두 반축
판정 규칙	**타원**, 3 샘플(60 ms) 연속	직사각형은 모서리에서 저속 지연이 생겼다
저마찰 트리거	제동 ≥ 0.3 이 0.3 s 지속 + 감속 < 1.2 m/s²	필터 지연 구간의 오탐 방지

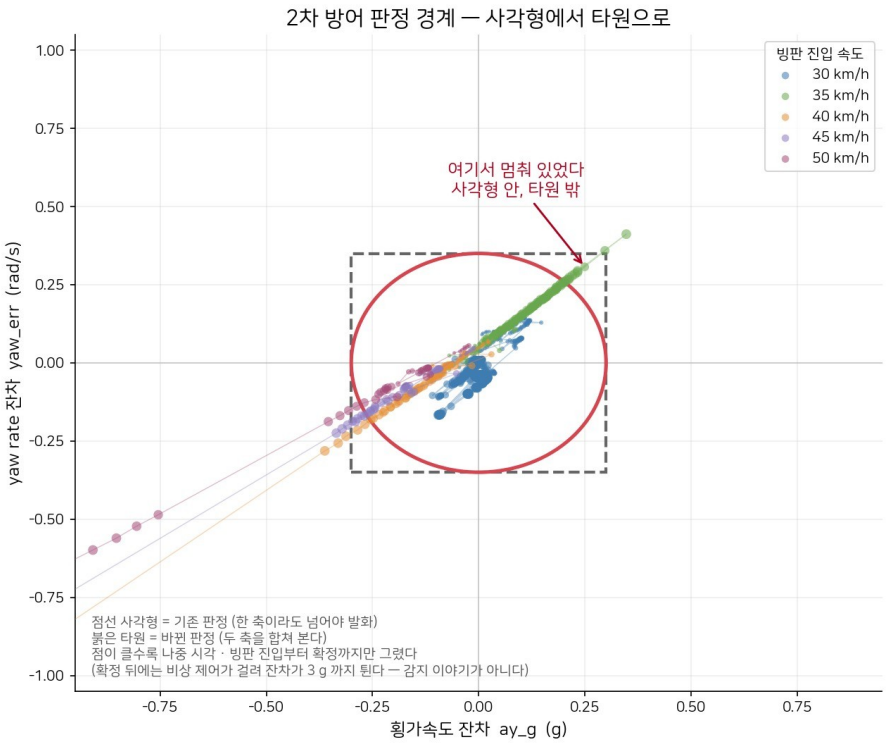


그림 10. 미끄러짐 판정 경계 — 직사각형 규칙의 모서리 지연을 없애려 타원 규칙으로 바꿨다

CARLA Town04, 맑은 낮, 40 km/h, 마찰 0.08, 1 차 방어를 끈 상태(미인식 가정)로 38.7 km/h 진입했다. 7.62 s 에 STM32N6 RTOS 가 미끄러짐을 확정했고(진입 1.78 s 만, a_y −0.364 g, yaw 오차 −0.282), hard_stop 뒤 왼쪽 차로가 비어 evade_left 로 전환해 11.68 s 에 차선 안에서 정지했다. 전체 2 차 방어 발동 62 회(보드 판정 53 · 호스트 9), 진입→확정 평균 2.41 s, 확정→정지 평균 3.78 s.

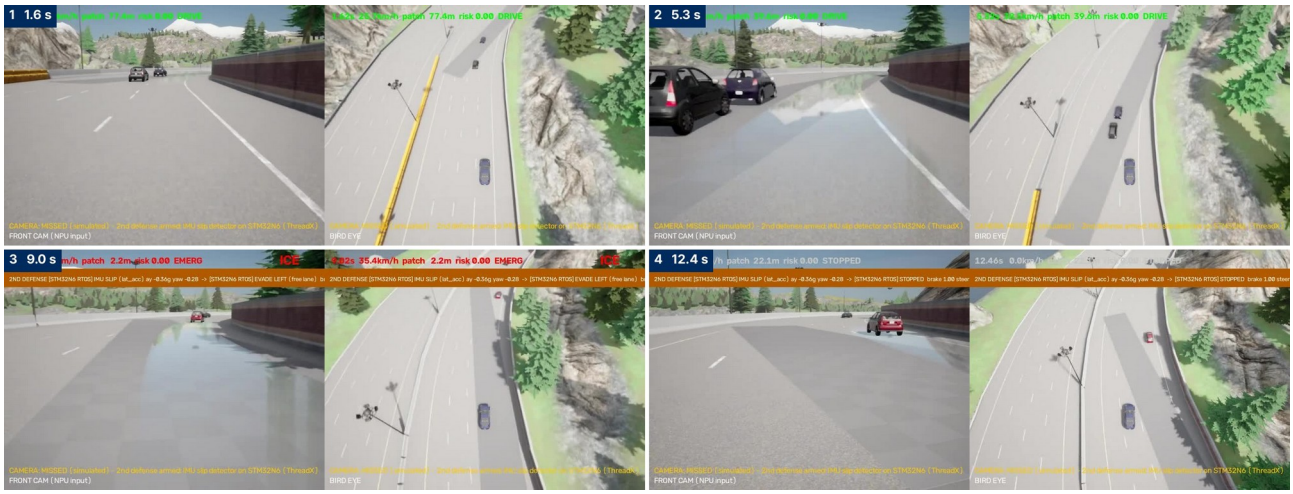


그림 11. 2 차 방어 동작 — 카메라가 놓친 빙판 진입 → STM32N6 이 미끄러짐 확정 → 차선 안 정지 (각 칸 왼쪽 전방 카메라, 오른쪽 조감)

C 코어(sw/fw/npu_lib/slip_core.h)는 HAL·OS 비의존이라 호스트에서도 컴파일되고, 파이썬 참조 구현과 같은 판정을 내리는지 매 커밋 검증한다.

6.2 왜 리눅스가 아니라 RTOS 인가

표 11. 같은 연산의 지연

플랫폼	표본	중앙값	최악
Pi 5 + Linux · 유휴	20,000	69 µs	**5,790 µs**
Pi 5 + Linux · 부하	20,000	68 µs	5,429 µs

STM32N6 + ThreadX (응답 전체)	49,405	12.4 µs	**22.6 µs**

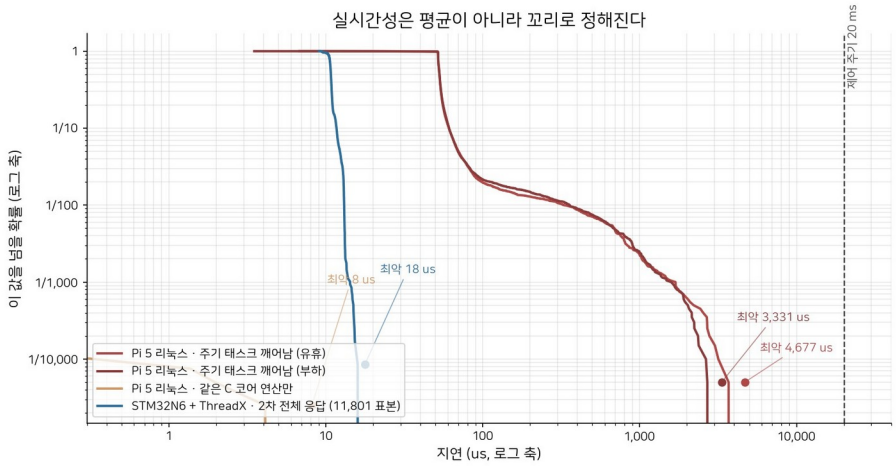


그림 12. 지연 분포(CDF) — 리눅스는 꼬리가 길고, ThreadX 는 최악 22.6 µs 안에 끝난다

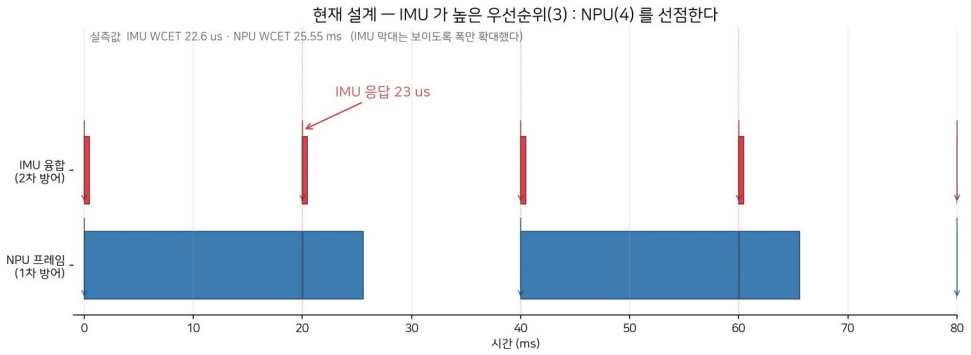


그림 13. RM 스케줄 — IMU 융합(우선순위 3)이 NPU 프레임(4)을 선점한다. 뒤집으면 25 ms 추론이 20 ms 주기를 막는다

리눅스는 평균이 빠르지만 꼬리가 길다. 안전 기능의 기준은 평균이 아니라 최악이다. 우선순위를 뒤집으면 25 ms 추론이 20 ms 주기를 막아 스케줄 자체가 불가능해진다.

6.3 방어가 없으면 / 2 차 방어가 있으면

1 차·2 차를 모두 끈 기준선 9 건은 전부 차선을 벗어나 88~99° 가로로 미끄러졌고, 5 건은 42~52 km/h 로 방호벽에 부딪혔다(충돌 직전 속도). 같은 조건에서 STM32N6 RTOS 2 차 방어는 미끄러짐을 확정하고 차선 안에서 세웠다. 2 차 방어 발동 62 회, 진입→확정 평균 2.41 s, 확정→정지 평균 3.78 s.

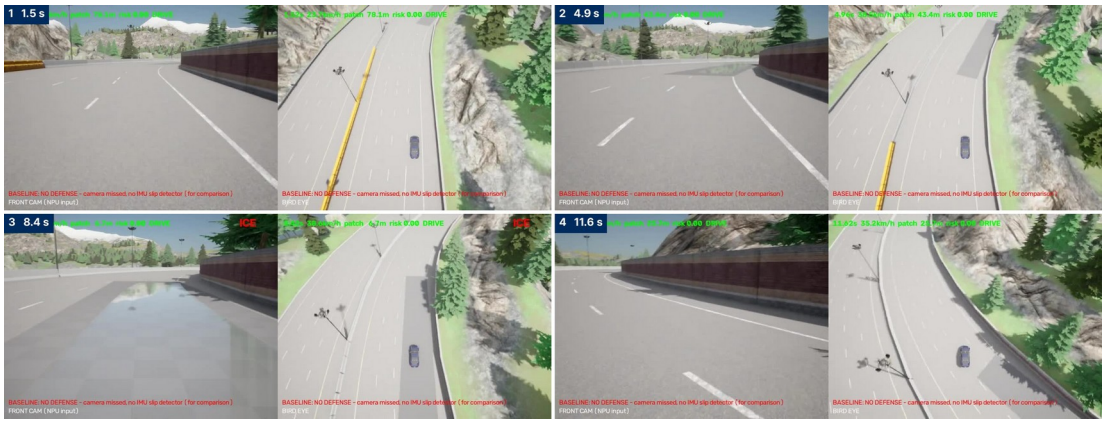


그림 14. 방어 없음 기준선 — 차선 이탈 → 가로 미끄러짐 → 42 km/h 로 방호벽 충돌 (화면 글씨 12 km/h 는 충격 뒤 속도)



그림 15. 같은 조건 직접 비교 — 각 칸 왼쪽 방어 없음, 오른쪽 STM32N6 RTOS 2 차 방어. 같은 날씨·주행선·빙판·주변 차량 8 대



그림 16. 날씨 4 종에서 같은 비교 — 각 쌍의 마지막 장면 (왼쪽 방어 없음, 오른쪽 RTOS 2 차 방어)

7. 2 차 방어 확장 — 노면 마찰을 재서 제동 시점을 고친다

7.1 '어떻게 밟나'가 아니라 '언제 밟나'

먼저 미끄러진 뒤의 제어를 다듬는 실험을 했다. 곡선 빙판에서는 마찰별 제어표 후보 78 개가 고정 게인과 충돌 직전 속도 중앙값 0.2 km/h 안으로 같았고, 직선 빙판 위 정지 차량 장면에서는 2 차 방어를 꺼도 결과가 같았다. 빙판에서는 누가 밟든 감속이 마찰 한계(μg)에서 막히고, 펄스 제동은 실차 ABS 가 이미 하는 일이다. 결과를 정한 것은 **제동을 시작한 시점**이었다(60 km/h 에서 9 s 전에 밟으면 전부 충돌, 13 s 전이면 정지).

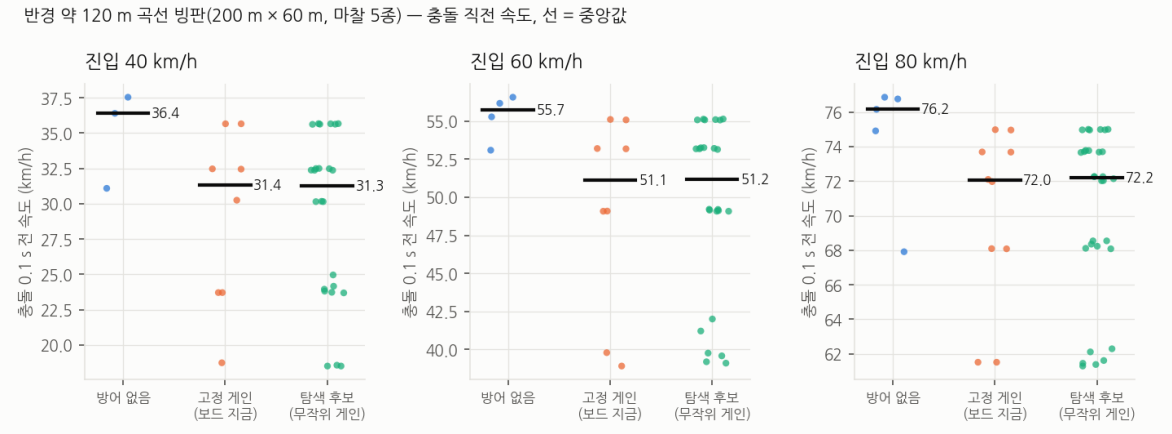


그림 17. 곡선 빙판 — 방어 없음 · 고정 게인 · 무작위 제어표 후보의 충돌 직전 속도. 후보는 고정 게인을 넘지 못했다

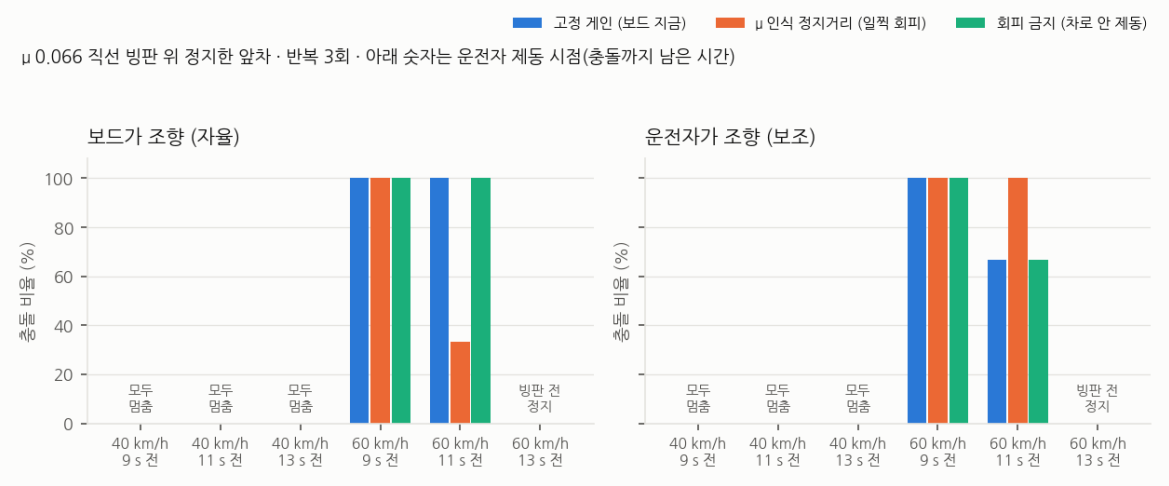


그림 18. 직선 빙판 위 정지 차량 — 판단 방식을 바꿔도 결과가 같다. 운전자 제동 시점(아래 숫자)이 결과를 정했다

표 12. 기존 기술과의 차이

	하는 일	못 하는 것
ABS · ESC	미끄러지는 바퀴를 잡고 차체 회전을 막는다	마찰을 다른 장치에 알리지 않는다
긴급제동 · 간격 유지	앞차 거리로 제동 시점을 정한다	마찰을 모르면 마른 노면 기준이라 빙판에서 늦다
노면 마찰 추정(일부 양산차)	바퀴 신호로 미끄러움을 추정해 경고·공유	제동 시점까지 바로 잇는 경우는 드물다
IcePredict 2 차 방어	**마찰을 직접 재서 제동 시작 시점을 노면에 맞춘다**	탭을 운전자가 느낀다

7.2 노면 확인 탭과 마찰 인식 제동 시점

빙판은 살짝만 밟아도 감속이 거기서 막힌다. 그 막힌 값이 곧 노면 마찰이다. 무거운 계산(마찰별 실측 감속 표)은 CARLA 에서 미리 끝내고, 보드는 20 ms 마다 표만 본다.

- **언제 확인하나** — ① 1 차 카메라 위험도가 회색 구간(0.3~0.6)에 0.2 s 이상이면 카메라가 본 지점(19 m 앞)에 도착했을 때, ② 영하 근처(3 °C 이하) 결빙 취약 구간 안이면 2.5 s 마다 최대 3 번. 안전 조건: 30 km/h 이상, 직진, 운전자 제동 없음, 앞차 여유.
- **노면 확인 탭** — 가속 페달을 0.2 s 떼고 브레이크 0.3 을 0.4 s. 마지막 0.25 s 평균 감속을 표로 되돌려 μ 를 잰다. 마른 노면이면 60 ms 만에 끝낸다(속도 손실 13 → 5 km/h).
- **20 ms 마다 판단** — 정지 거리 = 0.3 s × v + v^2 / (2 × 0.9 × 실측 감속) + 4 m 를 앞 물체 거리와 비교해 순항·가속 금지·예비 제동·전력 제동·정지 유지 중 하나를 고른다.

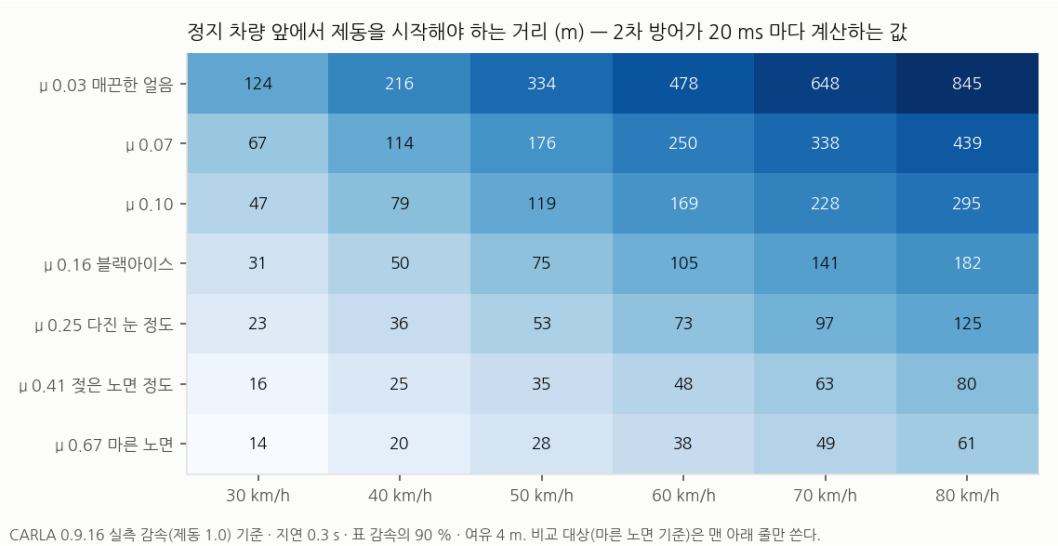


그림 19. 제동을 시작해야 하는 거리 — CARLA 실측 감속 기준. 60 km/h 마른 노면 38 m, 블랙아이스(μ 0.16) 105 m

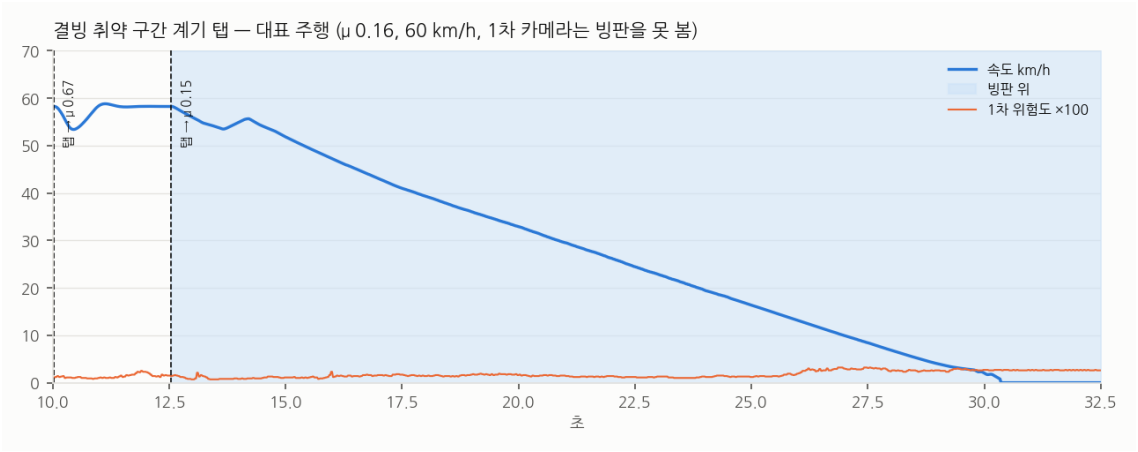


그림 20. 대표 주행 — 결빙 취약 구간 첫 탭은 마른 노면 '이상 없음', 다음 탭이 빙판 μ 0.15. 1 차 카메라 위험도는 내내 낮았다

7.3 CARLA 비교 결과

간격 유지 크루즈(ACC) 주행 중 빙판 위 정지 차량 장면에서, 같은 C 코드에 마찰 값만 다르게 넣었다. 비교 대상은 마른 노면 μ 를 가정하고, 2 차 방어는 탭으로 잰 μ 를 쓴다.

표 13. 마른 노면 기준 vs 마찰 인식 (마찰 3 종 × 속도 3 종)

	마른 노면 기준	마찰 인식
정지	2 / 9	**8 / 9**
충돌	7 / 9 (8 ~ 61 km/h)	1 / 9 (48 km/h, 필요 거리 295 m > 150 m)
정지 위치	—	7 회 4.1 ~ 4.2 m 앞 (목표 여유 4 m), 1 회 0.8 m 앞

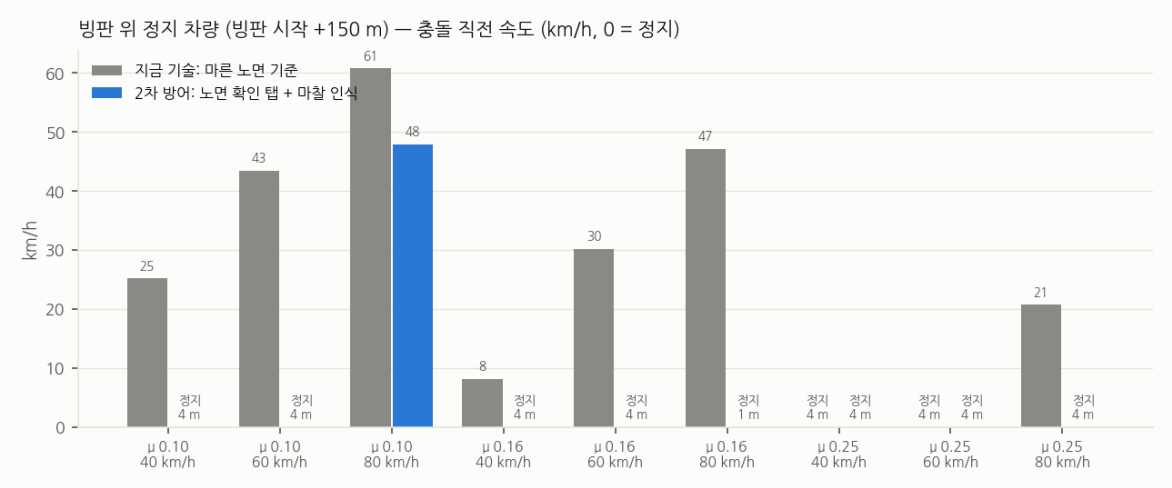


그림 21. 충돌 직전 속도(0 = 정지) — 마찰 3 종 × 속도 3 종



그림 22. 비교 장면 — 각 칸 위: 마른 노면 기준(30 km/h 충돌), 아래: 탭으로 마찰 측정(4.1 m 앞 정지)

표 14. 탭 판단 시험 (1 차 모델이 실제로 판정)

조건	결과
빙판 + 결빙 취약 구간 · 맑은 낮·밤·젖은 날씨	3 / 3 정지 — 첫 탭 마른 노면, 다음 탭에서 빙판
빙판 · 구간 정보 없음	카메라가 놓쳐 탭 없음 → 30 km/h 충돌
빙판 없음 · 구간 안 · 영하	탭 3 번 모두 '이상 없음' — 잘못된 제동 0
빙판 없음 · 영상 8 °C	탭 없음

1 차 카메라 모델은 이 CARLA 장면들에서 빙판을 거의 보지 못했다(위험도 최대 0.05). 그래서 카메라 계기만으로는 탭이 나가지 않았고, 지도의 결빙 취약 구간 정보가 그 공백을 메운다.

7.4 기초 작업 — CARLA 마찰 보정과 보드 마찰 추정기

마찰 인식 제동의 표는 CARLA 실측에서 나왔다. friction 9 종 × 시험 9 종으로 바퀴가 실제로 낸 최대 가속을 재서, friction 0.3 이하에서 실효 μ ≈ 0.81 × friction 임을 확인했다(0.5 부터는 제동 토크가 먼저 한계). 보드 C 코어에

는 미끄러짐 확정 뒤 펄스 제동 중의 합가속으로 μ 를 추정하는 기능(SLIP_MU_EST)을 넣었고, 실효 μ 0.41 이하에서 펄스 3 회(약 1 초) 안에 ±30 % 로 수렴했다.

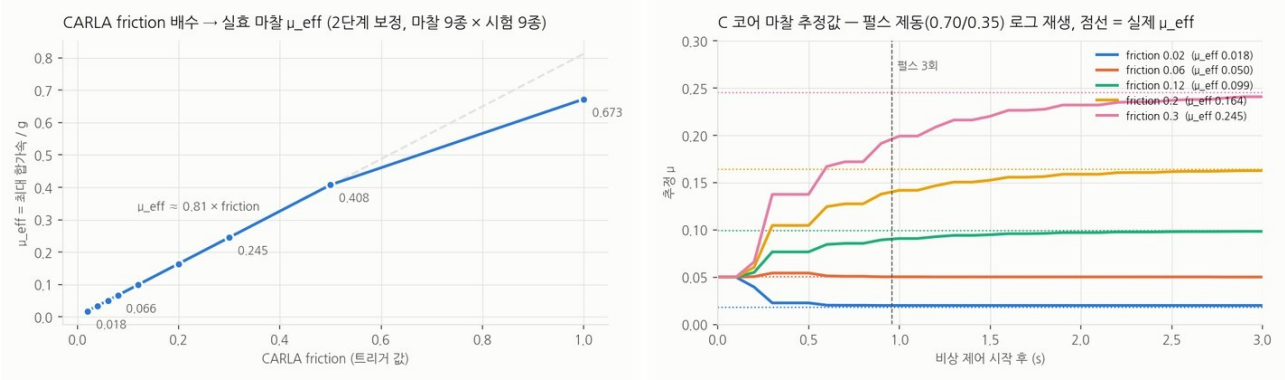


그림 23. 왼쪽: CARLA friction 과 실효 마찰(최대 가속/g). 오른쪽: 보드 마찰 추정기의 수렴 — 펄스 제동 로그 재생

이 과정에서 CARLA 결함 두 가지를 찾았다. 차량에 준 타이어 마찰 설정은 효과가 없어 마른 노면은 사실 기본값(횡 약 2 g)이었고, 빙판 트리거를 벗어나면 다시 그 값으로 돌아가 옆으로 미끄러지던 차가 2.5 g 로 급정지했다. 그래서 검증 주행은 빙판을 길고 넓게 깔고, 비교는 빙판 안 구간으로만 잘랐다.

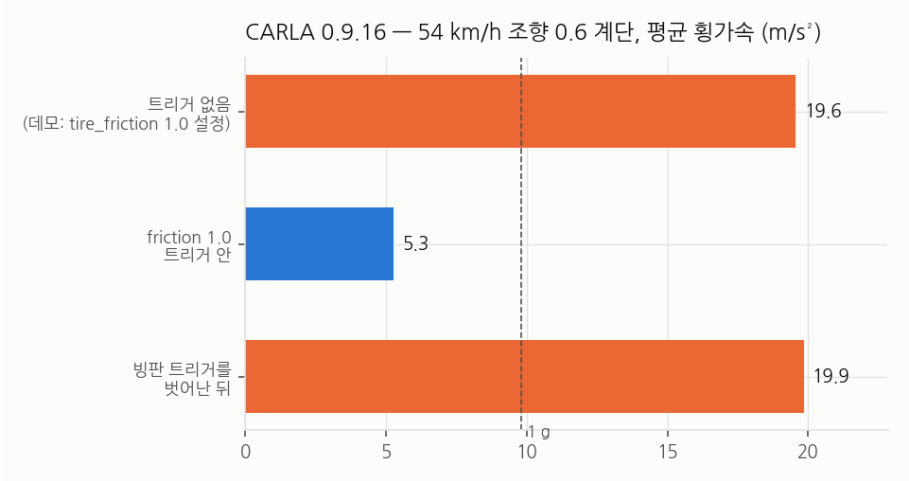


그림 24. CARLA 마찰 트리거 결함 — 트리거 밖으로 나가면 마찰이 기본값으로 돌아간다

측정 함정. CARLA 는 마찰 트리거 안에 다른 차량이 있으면 자차가 트리거 밖에 있어도 처음부터 그 마찰을 건다. 정지 차량을 트리거 밖에 두고 모든 실험을 다시 했고, 주행마다 출발 가속으로 마른 출발을 확인했다.

8. 안전 전용 RTOS — 메인 컴퓨터가 멈춰도 2 차 방어는 산다

STM32 의 몫은 '더 좋은 제어'가 아니라 **고장 격리**다. 곡선 빙판에서 메인 컴퓨터(파이)를 멈추자, 2 차 방어가 파이에 있을 때는 방어 없음과 같은 53.1 km/h 로, 안전 전용 RTOS 에 있을 때는 고장 없음과 같은 39.4 km/h 로 충돌했다. 파이 심장박동이 0.12 s 끊기면 축소 모드로 들어가 카메라·지도 입력을 버리고 이미 잰 μ 로 판단을 계속한다.

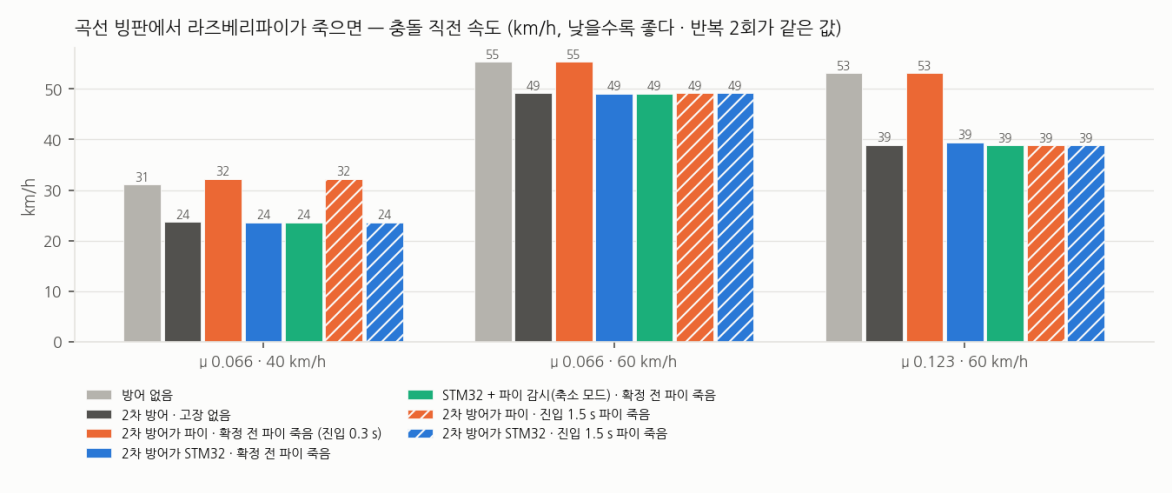


그림 25. 곡선 빙판에서 파이를 죽였을 때 충돌 직전 속도 — 2 차 방어 위치에 따라 결과가 갈린다

STM32N6 보드가 없어 확장 기능은 같은 C 코드를 **라즈베리파이 격리 코어**(코어 3 · SCHED_FIFO · mlockall, 안전 전용 RTOS 대체)에서 돌려 측정했다. 리눅스 커널 자체가 멈추면 같이 멈춘다 — 그래서 실차 설계는 별도 MCU 다.

표 15. 전 코어 부하에서 20 ms × 6,000 회

구성	평균 지연	최악 지연	마감 놓침
일반 리눅스 프로그램	2.6 ms	206 ms	211
실시간 우선순위 + 코어 고정	0.04 ms	0.23 ms	0
파이 격리 코어 (RTOS 대체)	0.018 ms	**0.06 ms**	**0**



그림 26. 빙판 확인 직후 파이 메인 프로그램을 실제로 강제 종료(SIGKILL) — 격리 코어의 2 차 방어가 혼자 4.1 m 앞에 세운다

9. 시연 구성

표 16. 시연 세 장면

장면	장비	보여주는 것
① 1 차 방어	RealSense D435i	빙판 사진을 비추면 1 차 모델이 실시간으로 '블랙아이스 감지 — 감속하

		세요'
② 2 차 방어	Moza 휠·페달 + CARLA	2 차 방어 OFF 면 평소처럼 밟아도 미끄러져 충돌, ON 이면 탭 → 경고 → 필요한 순간 자동 제동
③ 고장 격리	라즈베리파이 5	주행 중 파이 메인 프로그램을 실제로 죽여도 격리 코어의 2 차 방어가 계속 판단

사람이 운전해도 시뮬레이션은 실시간의 0.996 배로 돈다. 2 차 방어 판단은 파이 격리 코어가 20 ms 마다 내고, 캠퍼스 와이파이 왕복 지연은 중앙 4.8 ms 다. Moza 휠은 중앙 복귀 스프링, 탭 때 짧은 충격, 빙판 확정 때 진동으로 실차에서 몸으로 느끼는 감속을 대신한다.



그림 27. 왼쪽: 1 차 방어 시연 화면 — 카메라 입력과 노면 분류 확률·위험도·경고. 오른쪽: Moza 시연 화면 — 빙판 감지 배너, 2 차 방어 패널, 파이 메인 프로그램 정지 표시

10. 검증 방법과 재현성

· 실물 STM32N6 보드가 실제 펌웨어를 돌리고 CARLA 가 센서·물리를 제공하는 HIL 로 주행 108 회를 측정했다. 주행별 이벤트 JSON 을 저장소에 함께 올렸다.

· 1 차 성능은 시뮬레이션 화면이 아니라 실제 도로 사진 69,358 장을 보드에 넣어 쟀고, 장당 판정을 남겨 문턱을 바꿔 재계산할 수 있다.

· 2 차 방어 C 코드 = 파이썬 참조 = 파이 격리 코어 출력이 샘플 단위로 같다. 확장 기능은 컴파일 플래그(기본 0) 뒤에 있고, 끄면 작업 전과 출력이 비트 단위로 같다.

· 모든 수치는 저장소의 스크립트로 다시 만들 수 있다. 근거 문서 23 건, 결과 그림 41 건.

주행마다 무엇을 보고 판단했는지 남긴다. 시맨틱 라이다 64 채널에서 빙판 구간과 차량 3D 박스를, 카메라 영상에서 도로·차선·빙판·차량 2D 분할 라벨을 자동으로 만든다(AI Hub 형식 참고).

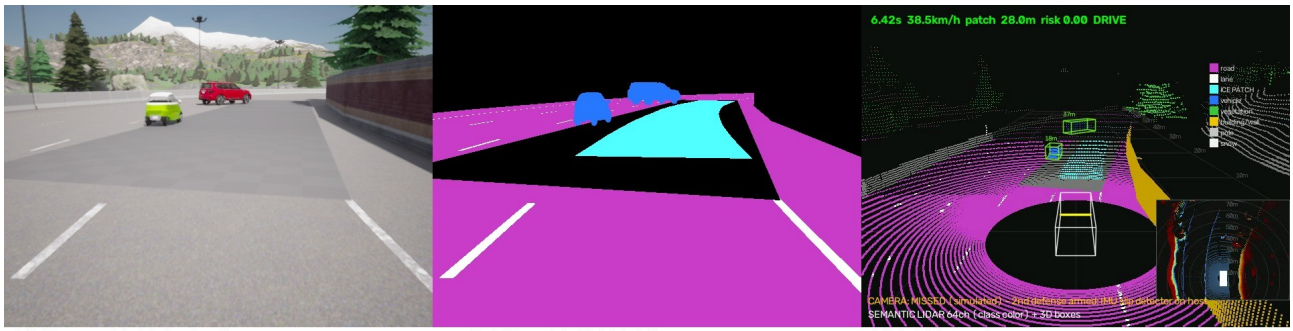


그림 28. 카메라 · 2D 분할 라벨 · 라이다 — 도로/차선/빙판/차량 라벨 자동 생성

11. 한계와 향후 계획

표 17. 한계

항목	내용
경보 거리	카메라 ROI 7.9 ~ 42.2 m. 모델 최소 80 px 과 ROI 해상도 91 px 이 경보 거리를 정한다 — 해상도를 올리는 것이 먼저다
2 차는 예방이 아니다	60 km/h 에서 1 차가 경고했지만 제동이 늦어 진입했고, 2 차가 0.26 s 만에 개입했는데도 39.2 km/h 로 앞차와 충돌했다
오경보·도메인 갭	밤에 41 m 밖 빙판을 두고 오경보가 났고, RSCD 질감을 입힌 CARLA 노면에는 같은 모델이 반응하지 않는다
탭	마찰 측정 탭을 운전자가 느낀다(마른 노면 5 km/h 손실)
시뮬레이션·측정	확장 결과는 CARLA 시뮬레이션이고, 실시간 측정은 파이 격리 코어 대체다. CARLA 타이어는 잠겨도 마찰이 줄지 않는다

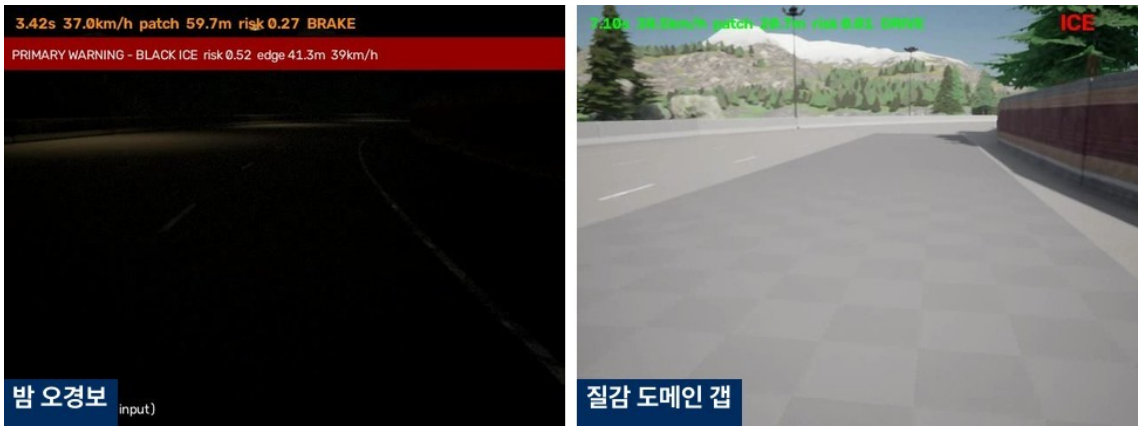


그림 29. 한계 사례 — 왼쪽: 41 m 밖 빙판을 두고 어두운 노면을 얼음으로 본 오경보(위험도 0.52). 오른쪽: RSCD 질감을 입힌 CARLA 노면에 모델이 반응하지 않았다

향후 계획

· 경보 거리 확대 — ROI 해상도를 올려 1 차가 보는 거리를 늘린다.

· 보드 실측 — 마찰 인식 제동 시점·파이 감시를 STM32N6 ThreadX 에서 다시 잰다.

· 운전자가 못 느끼는 마찰 측정 — 앞축 구동·뒷축 제동으로 차 전체 가속을 바꾸지 않고 마찰만 잰다.

· 놓친 장면 재학습 — 2 차 방어가 확정한 빙판 직전 카메라 영상을 1 차 방어 학습 데이터로 쓴다.

부록. 근거 문서 목록

표 18. 근거 문서 (저장소 sw/docs/evidence)

번호	문서
00	분석문서 목록
00	집계
01	탐지성능
02	융합가중치
03	도메인갭
04	질감의존
05	실사진 대규모평가
06	RTOS 가 왜 필요한가
07	노면조건별 분해
08	반사도헤드 전이검증
09	IMU 노이즈 허용예산
10	스케줄가능성 분석
11	변동스윕 2 차방어분포
12	저속지연 원인분해
13	판정규칙 타원

14	판정규칙 실주행 AB
15	맥락계층 기온게이트
16	조향지연보정
17	경보거리와 해상도
18	마찰별 게인스케줄링
19	안전전용 RTOS 연구계획
20	마찰인식 제동시점
21	시연 절차

소스 코드·근거 문서·영상: https://github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS